



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P.O.O : **Java**



Chapitre

5

Héritage et Polymorphisme

Sofiane HACHICHA
ISIMG 2025 - 2026

5- Héritage et Polymorphisme

5.1 Composition de classes (1/4)

P.O.O : Java



● Principe :

- La **réutilisation** par **composition** :
 - Quand une classe est créée, on peut l'utiliser aussi en tant que **nouveau type** dans une autre classe.
 - Les données d'une classe peuvent être des références (ou des instances) d'une autre classe.

```
class MaDate
{
    ....
}
```

```
class Personne
{
    private String nom;
    private String prenom;
    private MaDate naissance;
    ....
}
```

Classe String



Classe MaDate



Classe Personne



5- Héritage et Polymorphisme

5.1 Composition de classes (2/4)

POO : Java



Code source de la classe MaDate :

```
class MaDate {  
  
    private int j;    // jour  
    private int m;    // mois  
    private int a;    // an  
  
    public void initialise(int jour, int mois, int an)  
    { j=jour; m=mois; a=an; }  
  
    public MaDate( )  
    { this(1,1,1900); }  
  
    public MaDate(int j, int m, int a)  
    { this.j = j; this.m = m; this.a = a; }  
  
    public String toString()  
    { return j+"/"+m+"/"+a; }  
}
```



5- Héritage et Polymorphisme

5.1 Composition de classes (3/4)

P.O.O : Java



● Code source de la classe Personne :

```
class Personne{  
  
    private String nom;  
    private String prenom;  
    private MaDate naissance;  
    public Personne(String nom, String prenom, int j, int m, int a)  
    {  
        this.nom=nom;  
        this.prenom=prenom;  
        naissance=new MaDate(j,m,a);  
    }  
    public void affiche()  
    {  
        System.out.println("Identité Personne : ");  
        System.out.println(nom+ " " +prenom);  
        System.out.println(naissance);  
    }  
}
```



5- Héritage et Polymorphisme

5.1 Composition de classes (4/4)

POO : Java



● Code source de la classe TestPersonne :

- **TestPersonne** est la classe principale qui contient la méthode principale main().

```
public class TestPersonne {
    static{
        System.out.println ("Bloc statique");
    }

    public static void main(String args[ ])
    {
        Personne p = new Personne("X", "Y", 1,2,1999);
        p.affiche();
    }
}
```

Résultat d'affichage

```
Bloc statique
Identité Personne
XY
1/2/1999
```



5- Héritage et Polymorphisme

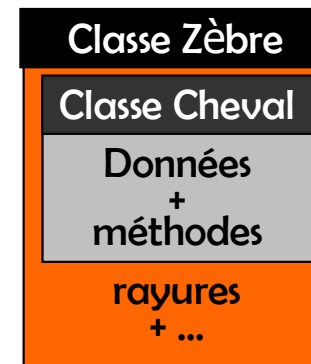
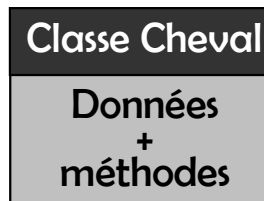
5.2 Héritage (1/14)

P.O.O : Java



● Principe :

- La **réutilisation** par **extension** :
 - Une classe, représentant un **concept voisin** d'une classe existante, n'a pas besoin d'être définie à partir de rien.
 - définition d'une classe comme extension d'une autre.
 - spécification des modifications seulement.
 - La deuxième classe **hérite** de la première.
 - première classe = **classe mère** ou **de base**
 - deuxième classe = classe **dérivée** (héritière) de la première





5- Héritage et Polymorphisme

5.2 Héritage (2/14)

POO : Java



● Intérêt :

- L'héritage **supprime**, en grande partie, les **redondances** dans le code.
- Une fois la hiérarchie de classes bien établie, on peut localiser en un point unique les sections de code (celles-ci restent à tout moment accessibles grâce au mot clé **super**).
- Si la hiérarchie des classes est bien pensée, on peut très facilement rajouter une classe, dans la mesure où on peut **réutiliser** le code des classes parentes.
- Si on n'a pas encore modélisé un comportement dans une classe donnée, et qu'on veut maintenant le rajouter, une fois l'opération terminée, ce comportement sera alors directement utilisable dans l'ensemble des sous-classes de celle considérée.



5- Héritage et Polymorphisme

5.2 Héritage (3/14)



- En Java, une classe B ne peut hériter que d'une seule classe A.

- Syntaxe :

```
class B extends A {
    ...
}
```

- Les classes dérivent, par défaut, de **java.lang.Object**.
- Le mot clé **super** représente l'objet parent (celui de la classe mère), de même que **this** représente l'objet courant.
- Une **référence** d'une classe A peut contenir **des instances de A** ou **des classes dérivées de A**.



5- Héritage et Polymorphisme

5.2 Héritage (4/14)

POO : Java



- L'opérateur **instanceof** permet de vérifier si une référence d'objet est une instance d'une classe ou d'une de ses sous-classes. Il prend en compte l'héritage, ce qui signifie que si une classe dérivée hérite d'une classe mère, l'opérateur renverra également true pour la classe mère.
- Une sous-classe peut **redéfinir** une ou plusieurs méthodes dont elle hérite.
 - La méthode **toString()** appartient initialement à la classe **Object**. Elle est donc héritée par toutes les classes. Il est possible de la redéfinir.
- Il est possible d'interdire qu'une méthode soit redéfinie dans une sous-classe en la précédant par le mot clé **final**.
- Il est possible d'interdire qu'une classe puisse être héritée en utilisant **final**.

5- Héritage et Polymorphisme

5.2 Héritage (5/14)



Exemple :

- Une classe mère « **Compteur** » qui permet de créer des compteurs classiques.
 - Elle contient une méthode d'incrémentation et une autre de décrémentation.

```
class Compteur {  
  
    private int valeur;  
  
    public Compteur( ) { valeur = 0; }  
    public void inc( ) {valeur ++; }  
    public void dec( ) {valeur --; }  
    public int getValeur( ) { return valeur; }  
  
}
```



5- Héritage et Polymorphisme

5.2 Héritage (6/14)

P.O.O : Java



Exemple :

- Une classe dérivée « **CompteurControle** » de la classe mère « **Compteur** » qui permet de créer des compteurs capables de contrôler l'incrémentation des compteurs classiques.

```
class CompteurControle extends Compteur
{
    private int maxval;
    public CompteurControle(int maxval)
    {
        super();
        this.maxval = maxval;
    }
    public void inc()
    {
        if( getValeur() < maxval ) { super.inc(); }
    }
    public int getMaxval() { return maxval; }
}
```

5- Héritage et Polymorphisme

5.2 Héritage (7/14)

P.O.O : Java



Exemple :

- **TestCompteur** est la classe principale.

```
class TestCompteur
{
    public static void main(String args[])
    {
        CompteurControle c = new CompteurControle(6);
        c.inc( );           // Première incrémentation
        c.inc( );           // Deuxième Incrémentation
        System.out.println("valeur : "+c.getValeur( ) );

        c.dec( );           // Première décrémentation
        c.dec( );           // Deuxième décrémentation
        System.out.println("valeur : "+c.getValeur( ) );

    }
}
```

Résultat d'affichage



5- Héritage et Polymorphisme

5.2 Héritage (8/14)



● Utilisation de « **super()** » :

```
public CompteurControle(int maxval) {
    super();
    this.maxval = maxval;
}
```

- Le constructeur de **CompteurControle** appelle le constructeur de la classe **Compteur** à l'aide **super()**.
 - **super()** : un appel au constructeur de la classe mère.
- Classiquement, l'appel du constructeur de la classe mère doit être la **première instruction** du constructeur de la classe dérivée.
 - En **Java 22** (en mode preview), une classe dérivée peut inclure des instructions dans son constructeur avant l'appel à `super()`. En **Java 23** (en mode preview), il est même possible d'initialiser des attributs avant l'appel à `super()`. En **Java 25** cette fonctionnalité passera en mode standard finalisé.

5- Héritage et Polymorphisme

5.2 Héritage (9/14)



● Utilisation de « **super** » et masquage des données :

```
public void inc()
{
    if( getValeur() < maxval ) { super.inc(); }
}
```

- Une partie du traitement de la méthode **inc()** de **CompteurControle** peut être effectuée par la méthode **inc()** de **Compteur**, c'est pourquoi **inc()** de **CompteurControle** appelle **inc()** de **Compteur** en utilisant **super.inc()**.
- **super** désigne l'instance en cours de la classe mère.
- La méthode **inc()** de la classe **CompteurControle**, **masque** (remplace) la méthode **inc()** héritée de la classe **Compteur**.

5- Héritage et Polymorphisme

5.2 Héritage (10/14)

P.O.O : Java



● Utilisation de « **super** » et masquage des données :

```
class A
{
    public int x;
    public void m( ) {...}
}
```

```
class B extends A
{
    public int x;
    public void m( ) {...}
}
```

```
class C extends B
{
    public int a;
    public void test(){
        a = super.x; // a reçoit la valeur de la variable x de la classe B
        a = super.super.x; // erreur syntaxique
        a = ((B) this).x; // a reçoit la valeur de la variable x de la classe B
        a = ((A) this).x; // a reçoit la valeur de la variable x de la classe A
        super.m( ); // Appel à la méthode m() de la classe B
    }
}
```



5- Héritage et Polymorphisme

5.2 Héritage (11/14)

P.O.O : Java



● Accès à des attributs **privés** de la classe mère :

```
class Compteur {
    private int valeur ;
    .....
}
```

```
class CompteurControle extends Compteur
{
    .....
    public void inc()
    {
        if( getValeur( ) < maxval ) { super.inc( ); }
    }
}
```

- L'attribut « **valeur** » est déclaré **private**.
 - Seules les méthodes de la classe **Compteur** ont accès à l'attribut « **valeur** ».
- Nécessité d'utiliser la méthode d'accès publique (l'accesseur ou getter) **getValeur()**.



5- Héritage et Polymorphisme

5.2 Héritage (12/14)

P.O.O : Java



- Accès à des attributs **protégés** de la classe mère :

```
class Compteur {
    protected int valeur ;
    .....
}
```

```
class CompteurControle extends Compteur
{
    .....
    public void inc()
    {
        if( valeur < maxval ) { super.inc( ); }
    }
}
```

- L'attribut « **valeur** » est déclaré **protected**.
- L'attribut « **valeur** » est hérité par la classe **Compteur_Control** et il est donc accessible par cette dernière classe.



5- Héritage et Polymorphisme

5.2 Héritage (13/14)

POO : Java



● Héritage : utilisation de « **instanceof** »

```
Compteur c1 = new Compteur( );  
CompteurControle c2 = new CompteurControle(1000);  
  
int maximum = c1.getMaxval( ) ; // Erreur  
System.out.println((c1 instanceof Compteur)); // true  
System.out.println((c1 instanceof CompteurControle)); // false  
System.out.println((c2 instanceof Compteur )); // true  
System.out.println((c2 instanceof CompteurControle)); // true
```



5- Héritage et Polymorphisme

5.2 Héritage (14/14)

P.O.O : Java



● Classe et méthode finale :

- une méthode déclarée **final** ne peut pas être redéfinie dans une classe dérivée.

```
class Base
{
    final void f()
    {---}
    ----
}
```

```
class Dérivée extends Base
{
    void f( )
    {---}
    ----
}
```

Erreur

- une classe déclarée **final** ne peut pas avoir de classes dérivées

```
final class Base
{
    ----
}
```

```
class Dérivée extends Base
{
    ----
}
```

Erreur



5- Héritage et Polymorphisme

5.3 Transtypage (1/2)

POO : Java



- Le transtypage, également connu sous le nom de conversion de type ou cast en anglais, consiste à faire passer un objet d'un type à un autre type. Cela peut impliquer des types de données primitifs (comme passer d'un int à un float) ou des objets dans le cadre de l'héritage (comme passer d'une référence de classe mère à une référence de classe fille).
- **Transtypage vers le haut, ou *upcasting* = implicite :**
Une référence d'une classe X peut contenir toute instance de la classe X ou de ses classes dérivées.

```
Compteur c1 = new Compteur( );  
CompteurControle c2 = new CompteurControle(1000);  
  
c1 = c2;      // transtypage vers le haut, upcasting  
  
System.out.println((c1 instanceof Compteur)); // true  
System.out.println((c1 instanceof CompteurControle)); // true
```

5- Héritage et Polymorphisme

5.3 Transtypage (2/2)

P.O.O : Java



● **Transtypage vers le bas, *downcasting* = explicite :**
Une référence d'une classe Y, dérivée d'une classe X, peut recevoir une référence de la classe X si et seulement si :

- L'objet référencé est bien du type Y ou d'un de ses types dérivés.
- Le changement de type (downcasting) vers le nouveau type doit être **explicite**.

```
...
c1 = c2;
// c2 = c1; // Erreur de compilation
c2 = (CompteurControle) c1; // transtypage vers le bas, downcasting
int maximum = ((CompteurControle) c1). getMaxval( ) ; // valide
```

- Avant d'effectuer un downcasting, il est recommandé d'utiliser **instanceof** pour éviter des exceptions de type `ClassCastException`. Cela permet de s'assurer que l'objet peut être converti en type souhaité sans erreur.



5- Héritage et Polymorphisme

5.4 Polymorphisme (1/6)

POO : Java



- Si une classe hérite une méthode de sa classe mère, alors il y a une chance de **redéfinir cette méthode** à condition qu'elle ne soit pas marquée final.
- La redéfinition d'une méthode héritée (en anglais, **overriding**) doit impérativement conserver la même déclaration que celle de la méthode de la classe mère
 - Il est recommandé d'utiliser l'annotation **@Override**, qui permet au compilateur de vérifier que le prototype de la méthode est correct.
- Quelle méthode est appelée ? → **Liaison dynamique**
 - Redéfinition la plus proche dans l'arbre d'héritage de la classe actuelle.
- Lors de la redéfinition d'une méthode, on ne doit pas restreindre sa visibilité.

5- Héritage et Polymorphisme

5.4 Polymorphisme (2/6)

POO : Java



● Exemple 1 :

```
class ClasseMere {  
    private final int x;  
    public ClasseMere(int x) { this.x = x; }  
    public int getX() { return x; }  
}  
class ClasseFille extends ClasseMere {  
    private final int y;  
    public ClasseFille(int x, int y) {  
        super(x);  
        this.y = y; }  
    public int getY() { return y; }  
    @Override  
    public int getX() { return super.getX()+1; }  
    @Override  
    public String toString() { return "["+super.getX()+" "+y+"]"; }  
}
```




5- Héritage et Polymorphisme

5.4 Polymorphisme (3/6)

P.O.O : Java



```
public class TestClasse {  
    public static void main(String[] args) {  
        ClasseFille o1 = new ClasseFille(1, 2);  
        System.out.println("(" + o1.getX() + ", " + o1.getY() + ")");  
        System.out.println(o1);  
  
        ClasseMere o2 = new ClasseFille(1, 2);  
        System.out.println(o2.getX());  
        System.out.println(((ClasseFille) o2).getY());  
        System.out.println(o2);  
    }  
}
```

Résultat d'affichage



5- Héritage et Polymorphisme

5.4 Polymorphisme (4/6)

P.O.O : Java



Exemple 2 :

```
class A {
    protected int f(int x) { return (x + 1); }
    public static int g(int x) { return (6); }
}

class B extends A {
    public int f(int x) { return (x + 2); }
    public static int g(int x) { return (x + 4); }
}

public class TestAB {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a instanceof B);
        System.out.println(a.f(2) * a.g(3));
    }
}
```

Résultat d'affichage





5- Héritage et Polymorphisme

5.4 Polymorphisme (5/6)

P.O.O : Java



Exemple 3 :

```
class A{  
    public String f (D obj ) { return "A et D"; }  
    public String f (A obj ) { return "A et A"; }  
}  
class B extends A{  
    public String f (B obj ) { return "B et B"; }  
    public String f (A obj ) { return "B et A"; }  
}  
class C extends B{ }  
class D extends B{ }  
  
public class TestABCD {  
    public static void main (String[ ] args) {
```





5- Héritage et Polymorphisme

5.4 Polymorphisme (6/6)

P.O.O : Java



```
A a1 = new A( );   A a2 = new B( );  
B b = new B( );   C c = new C( );   D d = new D( );  
System.out.println (a1.f(b));  
System.out.println (a1.f(c));  
System.out.println (a1.f(d));  
System.out.println (b.f(b));  
System.out.println (b.f(c));  
System.out.println (b.f(d));  
System.out.println (a2.f(b));  
System.out.println (a2.f(c));  
System.out.println (a2.f(d)) ;  
if( a1 instanceof B){  
    B b1 = (B) a1;  
    System.out.println (b1.f(b)) ; }  
}  
}
```

Résultat d'affichage

